Introduction to Deep Learning

Assignment 2: Generative Models and Sequence Modelling with PyTorch

November 2026

Task 1: Generative Models

The goal of this task is to learn how to train and use generative models for image-related tasks using **PyTorch**. You are provided with a starter Jupyter notebook (`A2_GenerativeModels.ipynb`) that illustrates how to build, train, and evaluate Convolutional Autoencoders (CAEs), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs).

State-of-the-art generative models can create high-fidelity photorealistic images (e.g., StyleGAN2 in Figure 1, and modern text-conditioned diffusion models). However, large generative networks are computationally expensive to train; therefore, starting with compact architectures allows us to study their core operating principles thoroughly.



Figure 1: Example output generated by StyleGAN2.

Your Tasks:

1. Study the provided notebook to understand how generative architectures are constructed in PyTorch using `nn.Module`:

   - **Convolutional Autoencoder (CAE):** Composed of a downsampling encoder (`ConvEncoder` with `nn.Conv2d`) and an upsampling decoder (`ConvDecoder` with `nn.ConvTranspose2d`).
   - **Variational Autoencoder (VAE):** Built similarly, but incorporates the *reparameterization trick* in the forward pass to sample latent vectors $z \sim \mathcal{N}(\mu, \sigma^2)$ differentiably, optimized using a joint reconstruction loss and analytical Kullback–Leibler (KL) divergence.

- **Generative Adversarial Network (GAN):** Consists of a generator and a discriminator trained with alternating gradient steps using separate optimizers (`torch.optim.Adam`).

2. Find a new image dataset of your choice online and use the code provided in the notebook to retrain your generative models (both VAE and GAN). Provide a link to your chosen dataset along with a concise description in your report.

3. To train the models reliably on your dataset, resize or crop your images to $64 \times 64 \times 3$. You may adapt the convolutional channel dimensions, kernel sizes, or depth in the PyTorch module definitions to best match your data.

4. Explore the learned continuous latent space:

   - Sample random latent vectors from the prior $\mathcal{N}(0, I)$ and feed them to the decoder/generator to create novel synthetic images.
   - Perform **latent space interpolation**: select two random latent points z_1, z_2 and generate images along the linear interpolation path $z(\alpha) = (1-\alpha)z_1 + \alpha z_2$ for $\alpha \in [0, 1]$ to visualize smooth transitions.

5. In your report, provide a detailed comparison of the working principles, loss functions, training stability, and sample quality between CAEs, VAEs, and GANs.

Task 2: Sequence Modelling with Recurrent Neural Networks

The goal of this task is to learn how to design and train encoder-decoder recurrent neural networks. Specifically, we investigate a sequence-to-sequence problem: teaching neural networks the underlying principles of basic integer arithmetic (addition and subtraction).

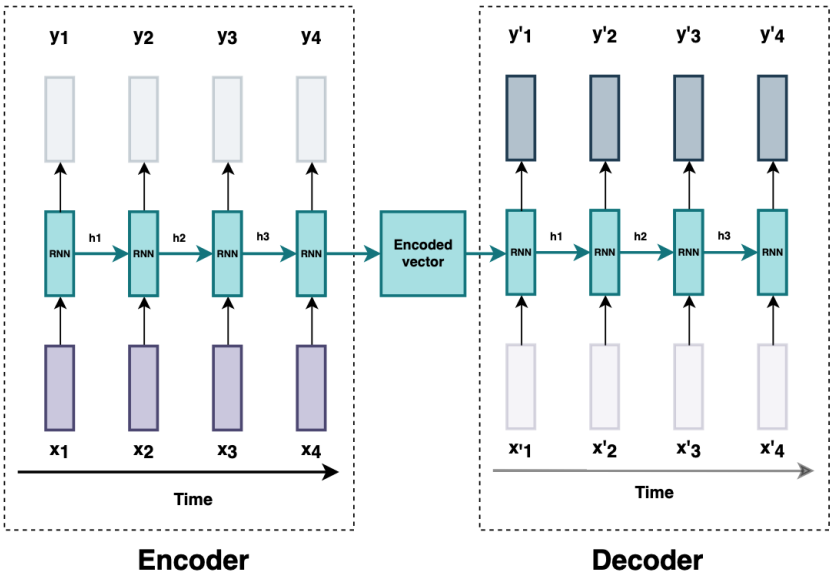


Figure 2: An illustration of an encoder-decoder recurrent network architecture.

Problem Description

Suppose we want a model that learns how to add or subtract two integers of up to two digits. Given an input query string of 5 characters (e.g., `'81+24'` or `'41-89'`), the network must output

a sequence of 3 characters representing the result (e.g., `'105 '` or `'-48 '`, padded with spaces). The character vocabulary consists of 13 tokens: digits 0–9, operators `+` and `-`, and space `' '`.

Beyond text strings, we can also explore multi-modal representations using the MNIST handwritten digit dataset by stacking images along the time dimension:



Figure 3: Converting query `'89+56'` and its answer `'145'` into sequences of images ($5 \times 28 \times 28$ query tensor, $3 \times 28 \times 28$ answer tensor) using MNIST samples.

You are provided with a starter notebook (`A2_RNNs.ipynb`) implementing synthetic arithmetic sequence generation and a reference PyTorch text-to-text sequence-to-sequence model using `nn.LSTM`.

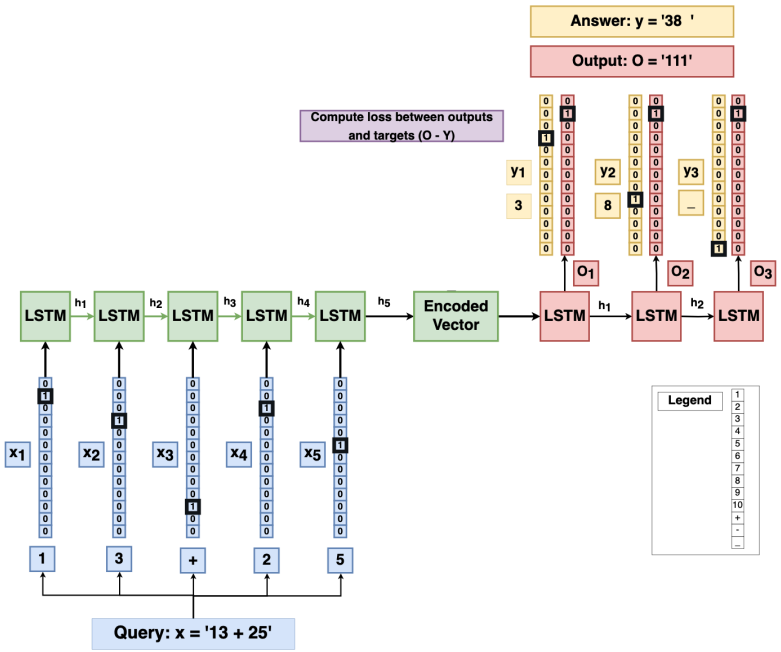


Figure 4: Recurrent sequence processing illustrated for arithmetic queries and answers.

Your Tasks:

1. Analyze the data generation pipeline and one-hot encoding routines. Inspect the provided PyTorch `Text2TextRNN` model, examining input/output tensor shapes and the role of the LSTM encoder and decoder.

2. **Generalization vs. Memorization:** Train the text-to-text model under different train/test split ratios (50%/50%, 25%/75%, 10%/90%). Evaluate character-level and sequence-level accuracy on unseen queries. Analyze and visualize common failure modes.

3. **Image-to-Text Model:** Build a PyTorch model that takes a sequence of MNIST images ($5 \times 28 \times 28$) and predicts the arithmetic answer as a text string (`'145'`). You can use a CNN or linear per-step feature extractor followed by an LSTM encoder.

4. **Text-to-Image Model:** Build a model that receives a text query string and generates a sequence of $3 \times 28 \times 28$ images corresponding to the answer digits. Use `nn.ConvTranspose2d`

in the image decoder and showcase sample generated sequences in your report.

5. **Multi-Layer Architectures:** Experiment with stacking multiple recurrent layers (`num_layers=2` in `nn.LSTM`) in the encoder and examine how depth impacts representation learning and generalization.

Deliverables

Your submission should consist of:

1. A report in PDF format (at most 10 pages).
2. Neatly organized and reproducible Jupyter notebook(s) with PyTorch code.
3. Exported `.py` files for each notebook.

Submit all files individually on Brightspace without compression (`.zip`/`.rar`).